

UNIT 1: FINITE AUTOMATA

Course Pattern: TE IT - 2024 Pattern (SPPU)	Academic Term: Semester V (2026)
Status: Comprehensive Master Theory Guide	Course Code: PCC-303-ITT

Syllabus Mapping & Unit Structure

The following table outlines the exam-oriented curriculum mapping and priority weightage for Unit 1.

Syllabus Topic	In-Depth Exam Coverage	Priority / Weightage
Basic Concepts	Symbols, Alphabet (Σ), Strings (w), String Length ($ w $), Empty String (ϵ), Formal Languages (L), Operations on Strings & Languages, Kleene Star (Σ^*), Positive Closure (Σ^+).	High Priority (4-6 Marks Qs)
Finite Automata (FA)	Formal 5-tuple definition ($Q, \Sigma, \delta, q_0, F$), State Transition Diagrams & Tables, Extended Transition Function ($\hat{\delta}$), Language acceptance by FA.	High Priority (6-8 Marks Qs)
NFA & ϵ -NFA	Definition of NFA, Transition function to power set $P(Q)$, NFA with ϵ -moves (ϵ -NFA), ϵ -closure computation algorithms.	High Priority (6-8 Marks Qs)
Inter-Conversions of FA	Conversion of ϵ -NFA to NFA (Eliminating ϵ -moves), Conversion of NFA to DFA (Subset / Power Set Construction), Conversion of ϵ -NFA directly to DFA.	High Priority (8-10 Marks Qs)
Minimization & Equivalence	Equivalence vs. Distinguishability of states ($p \equiv q$), Table-Filling Algorithm (Myhill-Nerode Theorem), Equivalence of two FAs.	High Priority (8-10 Marks Qs)
Output Machines (Transducers)	Moore Machine (6-tuple, state output, length $n + 1$), Mealy Machine (6-tuple, transition output, length n), Inter-conversions (Moore $\leftrightarrow$ Mealy).	High Priority (8-10 Marks Qs)
Applications of FA	Lexical Analyzer design in compiler construction (lex/flex), Pattern matching & string searching (KMP algorithm), Hardware logic controllers.	Medium Priority (4-6 Marks Qs)

Basic Concepts and Mathematical Foundations

Symbols, Alphabet, Strings, and Formal Languages

Automata theory is built upon formal mathematical structures representing discrete computation systems.

DEFINITION:

1. **Symbol:** An indivisible atomic entity (character, digit, or sign). Examples include $a, b, 0, 1, +$.
2. **Alphabet (Σ):** A finite, non-empty set of symbols.
 - Binary Alphabet: $\Sigma = \{0, 1\}$
 - English Alphabet: $\Sigma = \{a, b, c, \dots, z\}$
3. **String (Word):** A finite sequence of symbols chosen from an alphabet Σ .
 - String length $|w|$ denotes the total number of symbol occurrences in string w .
 - Empty String (ε or λ): The string containing zero symbols, with length $|\varepsilon| = 0$.
4. **Formal Language (L):** A set of strings formed from symbols of a given alphabet Σ , such that $L \subseteq \Sigma^*$.

Operations on Strings and Languages

Given an alphabet Σ , string operations form the operational substrate of state transitions:

1. **String Concatenation:** For $w_1 = ab$ and $w_2 = cd$, $w_1w_2 = abcd$. Length identity: $|w_1w_2| = |w_1| + |w_2|$.
2. **Identity Element:** For any string w , $w\varepsilon = \varepsilon w = w$.
3. **String Powers:** $w^0 = \varepsilon$, $w^1 = w$, $w^{\{k\}} = ww^{\{k-1\}}$.
4. **Prefix and Suffix:**
 - A string x is a **prefix** of w if there exists y such that $w = xy$.
 - A string y is a **suffix** of w if there exists x such that $w = xy$.
5. **Kleene Star Closure (Σ^*):** The set of all finite strings of any length over Σ , including the empty string ε :

$$\Sigma^* = \Sigma^0 \cup \Sigma^1 \cup \Sigma^2 \cup \dots = \bigcup_{i=0}^{\infty} \Sigma^i$$

6. **Positive Closure (Σ^+):** The set of all non-empty strings over Σ :

$$\Sigma^+ = \Sigma^1 \cup \Sigma^2 \cup \dots = \bigcup_{i=1}^{\infty} \Sigma^i = \Sigma^* - \{\varepsilon\}$$

NOTE:

- If $|\Sigma| = k$, then the total number of distinct strings of length n over Σ is k^n .
- Σ^* is countably infinite, whereas any individual string $w \in \Sigma^*$ is finite in length.
- A language L over Σ can be empty ($L = \emptyset$), finite ($L = \{00, 11\}$), or infinite ($L = \{a^n b^n : n \geq 0\}$).

Source: Hopcroft et al., Ch 1, pp. 43–45 | Source: Sipser, Ch 0, pp. 43–45

Finite Automata (FA) Models

A Finite State Machine (FSM) or Finite Automaton is a mathematical model of a system with discrete inputs, outputs, and a finite internal state memory.

Formal 5-Tuple Definition of a Deterministic Finite Automaton (DFA)

DEFINITION:

A Deterministic Finite Automaton M is defined as a 5-tuple:

$$M = (Q, \Sigma, \delta, q_0, F)$$

where:

1. Q is a finite, non-empty set of **states**.
2. Σ is a finite, non-empty set of **input symbols** (alphabet).
3. $\delta : Q \times \Sigma \rightarrow Q$ is the total **transition function**, mapping a state and input symbol to a unique next state.
4. $q_0 \in Q$ is the designated **initial state** (start state).
5. $F \subseteq Q$ is the set of **final states** (accepting states).

Transition Representations: Diagram and Table

An automaton can be represented equivalently via state transition diagrams or transition tables.

1. **State Transition Diagram:** A directed graph where:
 - Vertices represent states $q \in Q$.
 - Directed edges labeled with $a \in \Sigma$ represent transitions $\delta(q, a) = q'$.
 - The initial state q_0 is indicated by an incoming arrow pointing from nowhere.
 - Final states F are indicated by double concentric circles.
2. **State Transition Table:** A tabular layout where rows correspond to states and columns correspond to input symbols. The initial state is prefixed with $\rightarrow$ and final states with $*$.

Extended Transition Function ($\hat{\delta}$)

To formalize how a DFA processes entire strings rather than single symbols, we extend δ to $\hat{\delta} : Q \times \Sigma^* \rightarrow Q$ recursively:

- **Base Case:**

$$\hat{\delta}(q, \varepsilon) = q$$

- **Inductive Step:** For any string $w \in \Sigma^*$ and symbol $a \in \Sigma$:

$$\hat{\delta}(q, wa) = \delta(\hat{\delta}(q, w), a)$$

IMPORTANT:

Language Accepted by a DFA: The language $L(M)$ accepted by a DFA $M = (Q, \Sigma, \delta, q_0, F)$ is defined as the set of all strings in Σ^* that drive the automaton from initial state q_0 to some final state in F :

$$L(M) = \{w \in \Sigma^* : \hat{\delta}(q_0, w) \in F\}$$

A string w is **rejected** if $\hat{\delta}(q_0, w) \notin F$.

Source: Hopcroft et al., Ch 2, pp. 46–54 | Source: Sipser, Ch 1, pp. 46–52

Classification & Conversions of Finite Automata

Finite Automata are categorized based on non-determinism and transition rules.

Nondeterministic Finite Automata (NFA)

DEFINITION:

A Nondeterministic Finite Automaton M is defined as a 5-tuple:

$$M = (Q, \Sigma, \delta, q_0, F)$$

where Q, Σ, q_0, F are as defined for a DFA, but the transition function δ maps to the power set of states:

$$\delta : Q \times \Sigma \rightarrow P(Q) \quad (\text{where } P(Q) \text{ is the set of all subsets of } Q)$$

Extended Transition Function for NFA ($\hat{\delta}$):

- **Base Case:** $\hat{\delta}(q, \varepsilon) = \{q\}$
- **Inductive Step:** For $w \in \Sigma^*$ and $a \in \Sigma$:

$$\hat{\delta}(q, wa) = \bigcup_{p \in \hat{\delta}(q, w)} \delta(p, a)$$

NFA with ε -moves (ε -NFA)

An ε -NFA allows transitions without consuming input symbols (spontaneous moves):

$$\delta : Q \times (\Sigma \cup \{\varepsilon\}) \rightarrow P(Q)$$

DEFINITION:

For any state $q \in Q$, Epsilon-Closure(q) or ε -closure(q) is the set of all states reachable from q by taking zero or more ε -transitions.

Recursive Calculation Algorithm:

1. Base: $q \in \varepsilon$ -closure(q).
2. Induction: If $p \in \varepsilon$ -closure(q) and $r \in \delta(p, \varepsilon)$, then $r \in \varepsilon$ -closure(q).

For a set of states $S \subseteq Q$:

$$\varepsilon$$
-closure(S) = $\bigcup_{q \in S} \varepsilon$ -closure(q)

Conversion Algorithms

Algorithm 1: Conversion of ε -NFA to NFA (Eliminating ε -moves)

Given an ε -NFA $M = (Q, \Sigma, \delta, q_0, F)$, construct an equivalent NFA $M' = (Q, \Sigma, \delta', q_0, F')$:

1. Compute ε -closure(q) for every state $q \in Q$.
2. Define the new transition function δ' for state q and symbol $a \in \Sigma$:

$$\delta'(q, a) = \varepsilon$$
-closure($\delta(\varepsilon$ -closure(q), a))

3. Update the set of final states F' :

$$F' = \{q \in Q : \varepsilon$$
-closure(q) $\cap F \neq \emptyset\}$

Algorithm 2: Conversion of NFA to DFA (Subset Construction / Power Set Construction)

Given NFA $M_N = (Q_N, \Sigma, \delta_N, q_0, F_N)$, construct DFA $M_D = (Q_D, \Sigma, \delta_D, q_0^D, F_D)$:

1. Set the initial state of DFA: $q_0^D = \{q_0\}$. Add $\{q_0\}$ to Q_D as an unprocessed state set.
2. While there exist unprocessed state sets $S \in Q_D$:
 - For each input symbol $a \in \Sigma$:
 - Compute $S' = \bigcup_{q \in S} \delta_N(q, a)$.
 - Set $\delta_D(S, a) = S'$.
 - If $S' \notin Q_D$ and $S' \neq \emptyset$, add S' to Q_D as an unprocessed state set.

3. Define DFA final states $F_D = \{S \in Q_D : S \cap F_N \neq \emptyset\}$.

Algorithm 3: Direct Conversion of ε -NFA to DFA

1. Start with initial DFA state $q_0^D = \varepsilon\text{-closure}(q_0)$.
2. For each state subset $S \in Q_D$ and symbol $a \in \Sigma$:

$$\delta_{D(S,a)} = \varepsilon\text{-closure}\left(\bigcup_{q \in S} \delta(q, a)\right)$$

3. Repeat until no new state subsets are discovered. Set $F_D = \{S \in Q_D : S \cap F \neq \emptyset\}$.

IMPORTANT:

Equivalence Theorem: DFA, NFA, and ε -NFA are equivalent in language recognition power. Every NFA and ε -NFA can be converted to an equivalent DFA recognizing the exact same language L .

Source: Hopcroft et al., Ch 2, pp. 55–80 | Source: KLP Mishra, Ch 3, pp. 88–97

Minimization & Equivalence of Finite Automata

Minimization converts a DFA into an equivalent DFA with the minimum possible number of states.

State Equivalence and Distinguishability

DEFINITION:

1. **Equivalent States:** Two states $p, q \in Q$ of a DFA are **equivalent** ($p \equiv q$) if for all strings $w \in \Sigma^*$:

$$\hat{\delta}(p, w) \in F \iff \hat{\delta}(q, w) \in F$$

2. **Distinguishable States:** If there exists at least one string $w \in \Sigma^*$ such that one of $\hat{\delta}(p, w), \hat{\delta}(q, w)$ is in F and the other is not, then states p and q are **distinguishable** (denoted $p \not\equiv q$).
3. **k -Distinguishability:** States p and q are k -distinguishable if they can be distinguished by a string w of length $|w| \leq k$.

DFA Minimization Algorithm (Table-Filling Method / Myhill-Nerode Theorem)

1. **Unreachable State Elimination:** Remove all states that cannot be reached from the initial state q_0 via any path.
2. **Table Initialization:** Construct a triangular table for all pairs (p, q) with $p \neq q$.
3. **Base Step:** Mark all pairs (p, q) where $p \in F$ and $q \notin F$ (or vice versa) as distinguishable (check mark X).
4. **Inductive Step:** For each unmarked pair (p, q) and each symbol $a \in \Sigma$:
 - Find $p' = \delta(p, a)$ and $q' = \delta(q, a)$.
 - If pair (p', q') is already marked distinguishable, then mark (p, q) as distinguishable.
 - Repeat this process until an entire pass produces no new marked pairs.
5. **State Merger:** Group all remaining unmarked state pairs into equivalence classes using transitivity. Construct the minimized DFA where each equivalence class forms a single combined state.

Testing Equivalence of Two Finite Automata

To test whether two DFAs M_1 and M_2 are equivalent ($L(M_1) = L(M_2)$):

1. Combine M_1 and M_2 into a single merged transition table (renaming states to avoid collisions).
2. Run the Table-Filling Algorithm on state pairs.
3. If initial states $q_{\{01\}}$ and $q_{\{02\}}$ are found to be **equivalent**, then $M_1 \equiv M_2$; otherwise, they are not equivalent.

Source: Hopcroft et al., Ch 4, pp. 155–165 | Source: KLP Mishra, Ch 3, pp. 105–112

Finite State Machines with Output (Moore & Mealy Machines)

For transducer applications, finite state machines produce output strings during state transitions.

Moore Machine

DEFINITION:

A Moore machine is a 6-tuple:

$$M = (Q, \Sigma, \Delta, \delta, \lambda, q_0)$$

where:

- Q, Σ, δ, q_0 are as defined for a DFA.
- Δ is the finite **output alphabet**.
- $\lambda : Q \rightarrow \Delta$ is the **output function**, mapping each state to an output symbol.

Output Rule: The output depends **only** on the current state. For an input string $w = a_1 a_2 \dots a_n$ of length n , the output string length is $n + 1$:

$$\text{Output} = \lambda(q_0)\lambda(q_1)\lambda(q_2)\dots\lambda(q_n)$$

Mealy Machine

DEFINITION:

A Mealy machine is a 6-tuple:

$$M = (Q, \Sigma, \Delta, \delta, \lambda, q_0)$$

where $\lambda : Q \times \Sigma \rightarrow \Delta$ maps state-input pairs to output symbols.

Output Rule: The output depends on **both** the current state and the current input symbol. For an input string $w = a_1 a_2 \dots a_n$ of length n , the output string length is n :

$$\text{Output} = \lambda(q_0, a_1)\lambda(q_1, a_2)\dots\lambda(q_{n-1}, a_n)$$

Inter-Conversion Algorithms

Algorithm 1: Conversion of Moore Machine to Mealy Machine

Given Moore Machine $M = (Q, \Sigma, \Delta, \delta, \lambda_{\text{Moore}}, q_0)$: Construct Mealy Machine $M' = (Q, \Sigma, \Delta, \delta, \lambda_{\text{Mealy}}, q_0)$ with identical states:

$$\lambda_{\text{Mealy}}(q, a) = \lambda_{\text{Moore}}(\delta(q, a)) \quad \forall q \in Q, a \in \Sigma$$

Algorithm 2: Conversion of Mealy Machine to Moore Machine

Given Mealy Machine $M = (Q, \Sigma, \Delta, \delta, \lambda_{\text{Mealy}}, q_0)$:

1. For each state $q \in Q$, examine all incoming transitions to q .
2. If state q receives transitions producing different output symbols $b_1, b_2, \dots \in \Delta$, split state q into multiple sub-states $q_{\{b_1\}}, q_{\{b_2\}}, \dots$.
3. Assign output function $\lambda_{\text{Moore}}(q_b) = b$.
4. Re-route all transitions incoming to q producing output b to state q_b , and replicate all outgoing transitions of q for each split sub-state.

Source: KLP Mishra, Ch 3, pp. 84–88 | Source: Hopcroft et al., Ch 3, pp. 99–101

Real-World Applications of Finite Automata

Finite Automata serve as foundational computational structures across modern computer science disciplines:

1. **Lexical Analysis in Compilers:**

- Scanner generators (e.g. `lex`, `flex`) construct DFAs from regular expressions to tokenize program source code into identifiers, keywords, numbers, and operators.

2. **Pattern Matching and String Searching:**

- Algorithms like Knuth-Morris-Pratt (KMP) and Aho-Corasick construct deterministic state transitions to perform fast linear-time $O(n)$ pattern matching over text streams.

3. **Digital Logic and Controller Design:**

- Hardware sequential circuits, traffic light controllers, vending machines, and protocol stacks (e.g., TCP state machines) are modeled and verified using finite state transducers.

Source: Hopcroft et al., Ch 2, pp. 68–71 | Source: SPPU Syllabus PCC-303-ITT, p. 20

End of Theory of Computation Unit 1 Theory Notes — SPPU TE IT 2024 Pattern